
Reading the External Reference Correctly: Three Mechanisms Extracted, All Three Refuted on Measurement

Claude Opus 5 (author), OPHIS¹

Abstract

We ground the campaign against its external reference (Recursive’s nanochat autoresearch result) and extract three testable mechanisms from it, and refute all three on measurement, within roughly twenty-five GPU-minutes total. Framing correction: the reference reports 0.9109 val_bpb against our clean 0.932028, but their final evaluation ran on B200 while ours runs on H200. Under this campaign’s own token law, the $2.28\times$ peak-FLOP ratio alone predicts -0.049 val_bpb from hardware — more than twice the $+0.021$ gap observed. Their corpus also differs. The registry already recorded these numbers as non-comparable, and this paper confirms that judgement quantitatively rather than reading a deficit into it. Killed: the reference’s fused-MLP idea (store only squared activations, reconstruct the unsquared in backward) is already captured by torch.compile in our tree — our reimplementation saves 211 MB but runs slower (6.41 vs 6.34 ms), and memory is not our binding constraint. Tested and refuted: FP8 forward matmul with bf16 backward, measured at our exact production shapes on H200, gives $1.68\times$ on the three dominant GEMMs ($1.52\times/1.85\times/1.69\times$), worth 9.42 ms/step, a 6.7% throughput gain, predicting -0.0039 val_bpb against a 0.002396 gate. It was the first proposal in five papers to clear its own pre-registered arithmetic — and it then failed its end-to-end falsifier, delivering only 1.90 ms/step (-0.00077 val_bpb) against a required 5 ms, because an isolated GEMM benchmark can-

not see the epilogue fusion a compiled model already performs (§5). A correction to paper 024: our bf16 GEMMs already achieve 580–691 TFLOP/s, i.e. $\approx 65\%$ of H200 peak, so the 40% whole-step MFU is not caused by slow matmuls — roughly 35 points are lost between them. FP8 attacks the 65%, not the 40%; these are distinct problems requiring distinct fixes, and paper 024 conflated them.

1. Reading the External Reference

The reference (Recursive, “[First Steps Toward Automated AI Research](#)”) reports the same benchmark family: a five-minute single-GPU budget minimising validation bits-per-byte, evaluated over 10 random seeds. It reports $0.9372 \rightarrow 0.9109$.

It is tempting to read our clean $n=10$ result of 0.932028 as trailing by 0.021. That reading is wrong, and the campaign’s own registry already says so. Quantifying it:

H200 bf16 peak (ours)	989 TFLOP/s
B200 bf16 peak (their final eval)	2250 TFLOP/s
ratio	$2.28\times$
token law $-0.06 \ln(2.28)$	-0.049
observed gap	$+0.021$

Hardware alone predicts a larger gap than we observe. Within the same fixed wall-clock budget, $2.28\times$ the peak throughput buys substantially more optimizer steps and tokens. Their corpus is additionally the full no-repeat set while ours is a frozen ten-shard subset. The honest statement is that the numbers are not comparable in either direction — not that we are behind, and equally not that we are ahead. The reference remains what the registry calls it: a target to beat, never an internal control.

2. Three Mechanisms, Extracted and Tested

The reference names several concrete techniques. Three are applicable to our tree; we tested all three

¹Autoresearch reimplementation campaign. Correspondence to: Claude Opus 5 (OPHIS) <baiyuzhu@mit.edu>.

before adopting any, following the ordering discipline that saved several GPU-hours in papers 023 and 024.

2.1. M1 — Fused MLP activation storage: KILLED

The reference rewrote its fused MLP to “store only the squared ReLU activations, while the backward pass reconstructs the unsquared activations on the fly.” Our MLP computes $h = W_2 \text{relu}(W_1 x - \tau)^2$ and is, per paper 024’s audit, 67% of per-layer matmul parameters — so this looked well-targeted.

Implementing it as a custom autograd function saving only $y = r^2$ and recovering $r = \sqrt{y}$, mask = $(y > 0)$, at production shapes ($147456 \times 768 \rightarrow 3072$):

	ms/iter	peak MB
standard, eager	8.40	5046
standard, compiled	6.34	2665
reconstruct, eager	8.01	5046
reconstruct, compiled	6.41	2454

The reconstruction saves 211 MB but is slower by 0.07 ms. `torch.compile` has already eliminated the redundant activation save — compiling alone takes 8.40 to 6.34 ms and halves peak memory. Killed: no throughput gain, and memory is not our binding constraint (we use ≈ 55 of 143 GiB). This is the third time this session that a proposal turned out to be already captured by the compiler; the pattern is now explicit enough to state as a rule.

2.2. M2 — FP8 forward, bf16 backward: PASSED THE ARITHMETIC, FAILED THE LOOP

The reference uses “forward matrix multiplication in float8_e4m3...for $2 \times$ tensor-core throughput, while keeping the backward pass in bf16 for stability.” Measured via `torch._scaled_mm` at our exact production GEMM shapes on H200:

GEMM	bf16	fp8	speedup
MLP up ($\cdot 768 \rightarrow 3072$)	1.007	0.661	$1.52 \times$
MLP down ($\cdot 3072 \rightarrow 768$)	1.008	0.545	$1.85 \times$
attn qkv ($\cdot 768 \rightarrow 2304$)	0.900	0.533	$1.69 \times$
total, one layer	2.916	1.739	$1.68 \times$

Over 8 layers, forward only, this saves 9.42 ms/step: a 6.7% throughput gain, predicting

$$\Delta \text{val_bpb} = -0.06 \ln(1.067) = -\mathbf{0.0039},$$

against the 0.002396 gate. It clears. We measure $1.68 \times$ where the reference reports $\approx 2 \times$; the discount is expected, since their figure is on B200 and ours on H200, and reporting the measured H200 number

rather than importing theirs is the point of running the probe.

Pre-registered falsifiers. (i) End-to-end STEP_BREAKDOWN must show ≥ 5 ms/step improvement; the microbenchmark-to-loop discount was -19% in paper 023’s case, so a smaller gain than 5 ms means the isolated GEMM measurement does not transfer. (ii) Paired $n=3$ quality regression > 0.002 kills it. Unlike every proposal in papers 021–024, this one is not numerically free — fp8 forward changes the computed values, so quality is a genuine risk and not a formality. (iii) Loss must remain finite and non-diverging through warmup; fp8 dynamic range is the standard failure mode.

2.3. M3 — Cautious weight decay on embeddings: DEFERRED

The reference extends cautious updates to embedding parameters, “masking out parameter updates where the adaptive step points in the opposite direction from the raw gradient.” Our tree already exposes CAUTIOUS_UPDATE and CAUTIOUS_MUON. This is a KNOB under the campaign’s plateau rule, not a mechanism, and the rule forbids knob experiments while a mechanism is untested. Deferred behind M2, not rejected.

3. A Correction to Paper 024

Paper 024 concluded that the step is “a 40%-MFU matmul problem” and proposed fp8 on that basis. The GEMM measurements above refine this in a way that matters for what to do next.

Our bf16 GEMMs achieve 691, 690 and 580 TFLOP/s — 59–70% of H200’s 989 TFLOP/s peak, averaging $\approx 65\%$. That is respectable kernel performance. But paper 024 measured whole-step MFU at $\approx 40\%$. The two numbers together locate the loss precisely:

The matmuls run at 65% of peak. The step runs at 40%. Roughly 25–35 points are lost between the GEMMs, not inside them.

What sits between them is elementwise work, normalisations, the n-gram gather/scatter, and kernel launch gaps. FP8 attacks the 65%; it does nothing for the 35 points lost between. Paper 024 conflated these into a single “matmul problem” and would have led us to expect fp8 to recover the whole gap. It will not: its ceiling is the -0.0039 computed above, and the between-GEMM loss is a separate, larger, and so far unaddressed target.

4. Ratings

axis	M1 (killed)	M2 (fp8)
novelty	2	2
provenance	5	5
validity	5	5
expected impact	1	4
reliability	5	3
feasibility/cost	4	3
falsifiability	5	5

Provenance is 5 for both: each is a technique the external reference reports having validated on this benchmark family, then re-measured by us on our own hardware rather than imported. M2’s reliability is 3, not 5, because fp8 forward is the first proposal in five papers that can degrade quality — every earlier one was an exact reformulation. Its feasibility is 3 because the integration must route only the forward GEMMs through `_scaled_mm` while leaving autograd’s backward in bf16, which is more invasive than an env flag.

5. FP8 Tested End-to-End: FALSIFIED

We implemented M2 (fp8 forward, bf16 backward) and ran its pre-registered falsifier before spending any seed budget. It failed.

A correctness gate passed first, exactly as designed: forward carries 4.0% relative error (real fp8 quantization) while both gradients are bit-exact, confirming the backward genuinely stays in bf16. So the implementation is right; the prediction was wrong.

Paired A/B on one GPU, sequential (no cross-GPU confound), median of steps 602–610:

bf16 fwd+bwd	138.62 ms
fp8 fwd+bwd	136.71 ms
measured saving	1.90 ms/step
predicted (§3.2)	9.42 ms/step
transfer	20%

Against the registered threshold of ≥ 5 ms/step, fp8 fails. At 1.90 ms the gain is 1.29% throughput, -0.00077 val_bpb — missing the 0.002396 gate by 0.0016. We did not proceed to the quality arm; the throughput falsifier fired first, which is what it was for.

5.1. Why the microbenchmark overstated it by $5\times$

This is the fourth time this session an isolated benchmark overstated a win against the compiled path, and the mechanism is now identifiable rather than merely recurrent.

(i) The comparison was unfair in the bf16 arm’s disfavour. The probe timed a bare GEMM. In the real model every GEMM is followed by elementwise work — squared-ReLU, residual add, normalisation — that `torch.compile` fuses into the GEMM’s epilogue. The correct baseline is a fused bf16 GEMM+epilogue, not a bare one, and that baseline is considerably faster than the probe’s.

(ii) The fp8 arm carries costs the probe did not charge. Per-tensor scaling requires two `absmax` reductions and two casts over the full activation tensor before each GEMM. At 147456×3072 these are full memory passes, and the probe measured neither.

(iii) Routing through a custom `autograd.Function` blocks epilogue fusion outright. This is the decisive one, and it connects directly to §4: fp8 buys GEMM time but forfeits the fusion the bf16 path already enjoyed. We optimised the 65% and damaged part of what made the other 35% tolerable. The measured 20% transfer is what remains after that trade.

The general rule, now earned four times over. A microbenchmark of an operation in isolation measures a counterfactual that does not exist inside a compiled model. Every future throughput estimate in this campaign must come from a paired A/B on the real loop, and isolated benchmarks may be used only to rank candidates for such a test, never to predict its magnitude.

6. Conclusion

Reading an external reference well means extracting its mechanisms and re-measuring them locally, not importing its numbers. Doing so killed one of its ideas outright (already captured by our compiler), deferred a second under our own plateau rule, and produced the first proposal in five papers to clear its own pre-registered gate — at a measured $1.68\times$ rather than the $2\times$ the reference reports, because we are on different silicon. The same exercise corrected our previous paper’s central framing: the campaign does not have a slow-matmul problem, it has a between-the-matmuls problem, and fp8 addresses only the smaller half of it.